

Practical Deep Learning

11 September 2025

Prepared by Tanveer Hussain (Fellow HEA)
Lecturer, Edge Hill University
htanveer3797@gmail.com

Lecture Outline

- Data Handling
- Transfer Learning
- Designing CNN Model
- Practical

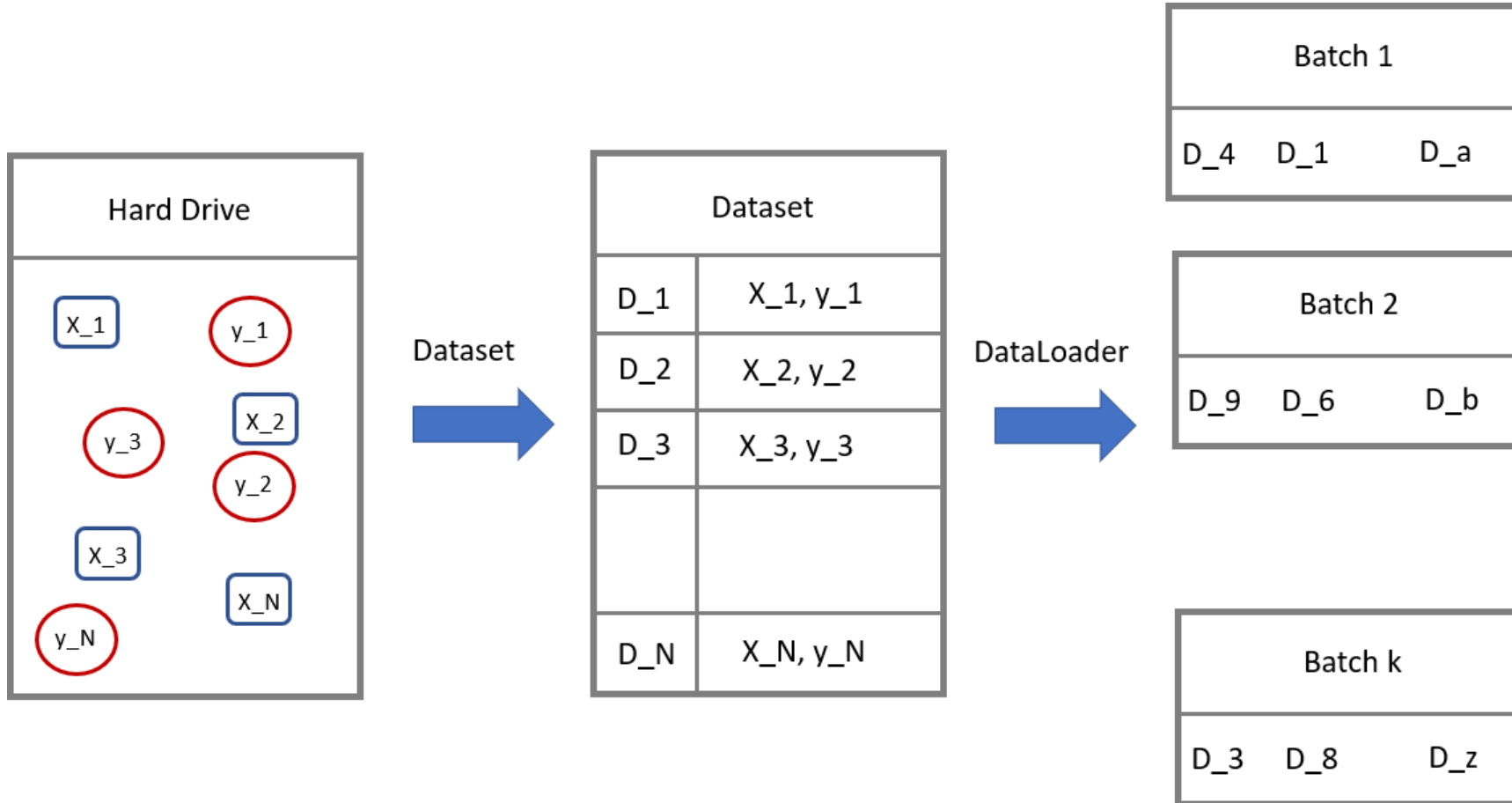
Data Handling

- As we have previously established that Deep Learning has been made possible by large amount of data and computational resource
- An important aspect to keep in mind is the data handling:
 - How do we handle large amounts of data?
 - How to we read different components of data (from possible different parts of our hard drive) and provide it to our training algorithms?
 - How do we feed this data to SGD algorithms in a streamlined manner?

Data Handling

- As we have previously established that Deep Learning has been made possible by large amount of data and computational resource
- An important aspect to keep in mind is the data handling:
 - How do we handle large amounts of data?
 - How to we read different components of data (from possible different parts of our hard drive) and provide it to our training algorithms?
 - How do we feed this data to SGD algorithms in a streamlined manner?
- PyTorch provides Dataset and DataLoaders to handle data in an efficient manner.
- We will extend the Dataset and DataLoaders class to construct our own Dataloaders.

Data Handling





Data Handling

```
class CustomDataset(Dataset):
    """ CustomDataset dataset."""

    # Initialize your data, download, etc.
    def __init__(self):
        xy = torch.load('data-datasets.csv', delimiter=',', dtype=torch.float32)
        self.len = xy.shape[0]
        self.x_data = torch.from_numpy(xy[:, 0:-3])
        self.y_data = torch.from_numpy(xy[:, -3])

    def __getitem__(self, index):
        return self.x_data[index], self.y_data[index]

    def __len__(self):
        return self.len

dataset = CustomDataset()
train_loader = DataLoader(dataset=dataset,
                           batch_size=32,
                           shuffle=True,
                           num_workers=2)
```

Data Handling



```
dataset = CustomDataset()
train_loader = DataLoader(dataset=dataset, batch_size=32, shuffle=True, num_workers=2)

# Training Loop
for epoch in range(2):
    for i, data in enumerate(train_loader, 0):
        # get the inputs
        inputs, labels = data

        # wrap them in Variable
        inputs, labels = Variable(inputs), Variable(labels)

        # Forward pass: Compute predicted y by passing x to the model
        y_pred = model(inputs)

        # Compute and print Loss
        loss = criterion(y_pred, labels)
        print(epoch, i, loss.data[0])

        # Zero gradients, perform a backward pass, and update the weights.
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Transfer Learning

- Practical Implementation of Deep Learning algorithms is just as much an art as it is a science.
- The main takeaways if not to start from scratch rather to build on top of the previous knowledge.
- Today, we will look at some important tools used in the practical implementation of Deep Learning algorithms.

Transfer Learning

- Improvement of learning in a new task through the transfer of knowledge from a related task that has already been learned.
- We will look at one strategy of transfer learning called Fine-Tuning

When to Fine-tune?

1. New dataset is small with distribution similar to original dataset.
 - Keep the feature extraction part fixed and fine-tune the classifier part of the network

When to Fine-tune?

1. New dataset is small with distribution similar to original dataset.
 - Keep the feature extraction part fixed and fine-tune the classifier part of the network
2. New dataset is large with similar distribution to the original dataset
 - Fine tune both the feature extractor and the classifier part of the network

When to Fine-tune?

1. New dataset is small with distribution similar to original dataset.
 - Keep the feature extraction part fixed and fine-tune the classifier part of the network
2. New dataset is large with similar distribution to the original dataset
 - Fine tune both the feature extractor and the classifier part of the network
3. New dataset is small but different distribution from the original dataset
 - Use SVM classifier on the features extracted from the feature extractor part of the Network



When to Fine-tune?

1. New dataset is small with distribution similar to original dataset.
 - Keep the feature extraction part fixed and fine-tune the classifier part of the network
2. New dataset is large with similar distribution to the original dataset
 - Fine tune both the feature extractor and the classifier part of the network
3. New dataset is small but different distribution from the original dataset
 - Use SVM classifier on the features extracted from the feature extractor part of the Network
4. New dataset is large and different distribution from the original dataset
 - Fine tune both the feature extractor and the classifier part of the network

Designing Data-Specific CNN Model



References

- https://web.eecs.umich.edu/~justincj/slides/eecs498/WI2022/598_WI2022_lecture02.pdf
- https://web.eecs.umich.edu/~justincj/slides/eecs498/WI2022/598_WI2022_lecture03.pdf
- <https://towardsdatascience.com/training-deep-neural-networks-9fdb1964b964>
- <https://www.charles-deledalle.fr/pages/teaching.php>
- https://www.charles-deledalle.fr/pages/files/ucsd_ece285_mlip/3_deep.pdf